

INTRODUCTION

Link to Github Repository: <https://github.com/konradfuniwue/PracticalDataScience>

Link to Presentation: [Datascience CVs.pptx](#)

Dominik Ries did EDA & Baseline Models (Rules-Based, TF-IDF)

Konrad Fendt did OOD & Data Manipulation

Julius Alliger did Bert and Ensemble Implementation (Training and Testing) & Data Manipulation

This documentation outlines the collaborative work behind the development of the snapADDY Capstone Project. The following sections detail individual contributions, including Exploratory data analysis, our used Methodologies, and evaluation. This record serves both as recognition of the team's efforts and as a transparent account of the model's development process.

EXPLORATORY DATA ANALYSIS (EDA)

Purpose and scope of the notebook

The first notebook, `main_eda.ipynb`, focuses on exploratory data analysis and on validating the evaluation setup. Its main purpose is to ensure that the datasets and the evaluation logic are well understood before any models are trained. The notebook does not aim to optimize models or tune hyperparameters. Instead, it answers fundamental questions about the data, such as the size and structure of the training datasets, the degree of label imbalance, the typical length and ambiguity of job titles, the presence of multilingual data, and the consistency between training labels and evaluation labels.

This step is essential because model performance is strongly constrained by data quality and by the evaluation design. If the evaluation is poorly defined, even strong models can produce misleading results. This notebook helps to avoid common mistakes such as evaluating multiple jobs per CV without a clear rule, comparing models on different subsets of the data, over-interpreting high accuracy caused by a dominant label, or ignoring label space mismatches between training and evaluation data.

1. Setup and 2. Data Paths

After a short setup section with imports and display configuration, the notebook defines all relevant file paths. This ensures that the notebook can be executed reproducibly from the project root without manual adjustments.

3. Load Labeled CSVs (training)

Two labeled CSV files act as training data, one for department prediction and one for seniority prediction. Both contain a simple supervised format with a text column and a label column. In this section, the CSVs are loaded and inspected to verify that the expected columns exist, that there are no accidental index columns, and that dataset sizes and label counts look reasonable. This establishes a clear understanding of what the models will actually be trained on and serves as a first sanity check of the training data.

4. Label balance (training CSVs)

Label distributions are analyzed to understand class imbalance for both department and seniority. This step is important because imbalance can inflate accuracy while hiding poor performance on minority classes. Extremely rare labels are also problematic because their evaluation metrics tend to be unstable.

The conclusions from this section directly motivate the later use of macro-averaged metrics instead of relying on accuracy alone.

5. Text sanity checks (training CSVs)

The notebook then performs basic text sanity checks, focusing mainly on job title length. Many titles are very short, which naturally increases ambiguity, especially for department classification where vocabulary overlaps across domains. This observation already sets realistic expectations for later model performance.

6. Languages in the CSVs (quick heuristic)

A lightweight multilinguality check is included to estimate whether multiple languages or scripts are present in the data. While this is not a full language detection pipeline, it provides a useful signal that multilingual data may exist. This matters because multilinguality can negatively affect token-based models, especially if training and evaluation distributions differ.

7. Train/validation split sanity check

Another important section validates the train-validation split used later in modeling. A stratified split is checked to ensure that label proportions are preserved and that rare labels are not accidentally removed from the validation set. This prevents misleading internal evaluation results and ensures that reported metrics are meaningful.

8. JSON reality check (evaluation data)

The most critical part of main eda.ipynb concerns the evaluation setup for the JSON data. The JSON file represents a more realistic deployment scenario, as each CV can contain multiple job entries.

Since each CV can contain multiple jobs, the notebook defines a deterministic rule to select exactly one “current job” per CV. Only jobs marked as ACTIVE are considered. If multiple active jobs exist, the one with the most recent start date is selected. If dates are missing, the original job order is used as a fallback. This rule matches the project definition of a current job, handles edge cases, and ensures reproducibility.

The notebook also examines the resulting label distributions in the JSON evaluation data and compares them to the training distributions.

Label space consistency and evaluation validity

checks for label space mismatches between the training data and the JSON evaluation data, especially for seniority. If the JSON contains labels that do not appear in the training CSVs, a supervised model cannot learn to predict them. Identifying such mismatches is crucial, because errors caused by unseen labels should not be interpreted as model failures.

Summary and implications for modeling

Overall, the outputs of main eda.ipynb are not model quality results. Instead, they provide evidence about the data and the validity of the evaluation setup. High imbalance motivates the use of macro F1 later, short titles explain unavoidable confusion, multilinguality suggests possible vocabulary mismatch, and differences between CSV and JSON distributions explain why realistic evaluation is usually harder.

The notebook therefore sets clear and realistic expectations for the baseline modeling results that follow.

Data Manipulation

Originally, our approach focused on unifying the dataset for an English-centric model, as our early architectures were not pretrained on multilingual tasks. To achieve this, we implemented a translation pipeline using the NLLB (No Language Left Behind) model from Hugging Face.

The process followed a three-stage pipeline:

1. **Language Detection & Quality Control:** Before translation, data underwent a labeling phase using a pre-trained transformer (papluca/xlm-roberta-base-language-detection). This model assigned a language label and a confidence score to each entry.
2. **Strict Filtering:** To ensure high translation quality, we applied a filtering threshold. Entries with a confidence score below 0.85 were discarded. Furthermore, we restricted the dataset to specific high-resource languages, removing any columns labeled outside of German (de), English (eng), and French (fr).
3. **Linguistic Unification:** Once filtered, the NLLB model was used to translate the remaining diverse entries into a single target language.

While this pipeline successfully unified the data, subsequent evaluations revealed that newer multilingual models inherently outperformed our English-centric versions. As these models demonstrated a superior ability to handle diverse linguistic inputs without preprocessing, we eventually moved away from using these manual translations in favor of native multilingual training.

BASELINE

The notebook Baseline_models.ipynb implements clean, reproducible baseline models for predicting Department and Seniority from job titles. Unlike the EDA notebook, it does not analyze the data in depth again. Instead, it focuses on model training, evaluation, and interpretation of results.

The goal of this notebook is not to achieve maximum performance, but to establish strong and transparent reference points. These baselines define a lower and mid-level performance range against which more advanced models can later be compared.

1. Setup and 2. Data paths

After basic imports and configuration, the notebook defines all required input paths. These include the two labeled CSV files used for training and the annotated JSON file used for realistic evaluation. Centralizing paths ensures that the notebook can be run reproducibly from the project root.

3. Load labeled CSVs

In this section, the department and seniority CSV datasets are loaded. Each dataset follows a simple supervised format with text and label columns. Basic previews and dataset sizes are printed as a fast sanity check to confirm that the expected data format is used for training.

4. Train test splits

For both tasks, a stratified train–test split is created. Stratification ensures that label proportions are preserved in both splits and that rare labels are not accidentally removed from the test set. This step guarantees that CSV-based evaluation is fair and reproducible.

5. Load JSON evaluation data

This section constructs the evaluation datasets based on the annotated JSON file. The logic mirrors the evaluation design defined earlier in `main.eda.ipynb`, where exactly one current job per CV is selected. Two JSON evaluation sets are created:

- Department evaluation rows
- Seniority evaluation rows with a strict label space

For seniority, rows with labels that do not exist in the training CSVs are filtered out. This strict handling ensures that evaluation only includes labels the model could theoretically learn.

6. Baseline 1: Rule based

The first baseline family is a rule based approach. When an exact title match is found in the training data, the corresponding label is predicted directly. If no exact match exists, the model predicts based on token overlap between the job title and label-specific token sets derived from the training data.

This baseline is evaluated separately for both tasks:

- Department evaluation on the CSV test split and on JSON current jobs
- Seniority evaluation on the CSV test split and on JSON current jobs with strict label space

After each confusion matrix, a short interpretation explains the dominant error patterns. This makes the behavior of the baseline transparent and easy to reason about.

7. Baseline 2: TF IDF plus Logistic Regression

The second baseline uses a classical machine learning approach. Job titles are converted into TF-IDF features using unigrams and bigrams. A Logistic Regression classifier is then trained, with class weighting applied to reduce the effects of label imbalance.

The evaluation structure mirrors the rule based baseline:

- Training on the CSV train split (or full CSV for JSON evaluation)
- Evaluation on the CSV test split
- Evaluation on JSON current jobs

For seniority, strict JSON evaluation is again applied to avoid unseen labels.

This baseline is particularly appropriate because job titles are short and often contain strong lexical cues, making n-gram based features effective.

8. Summary table

This section aggregates the main results into a single table. Accuracy and macro F1 are reported for all combinations of:

- Baseline approach (rule based vs TF-IDF + Logistic Regression)
- Task (Department vs Seniority)
- Evaluation set (CSV test vs JSON current jobs)

9. Key findings

The notebook concludes with a short textual summary of the main observations. This section highlights consistent patterns across models and evaluation settings and can be reused almost directly as a “Key Findings” subsection in the report.

Metrics and outputs

For each evaluation, the notebook reports multiple complementary outputs:

- Overall metrics (accuracy and macro F1)
- Per-class precision, recall, F1 score, and support
- Confusion matrices with absolute counts
- Tables of the most frequent misclassifications

These outputs provide both a high-level performance view and detailed insight into systematic errors.

Macro F1 is emphasized because it treats all classes equally and is therefore more informative under class imbalance than accuracy alone. Confusion matrices are especially useful for identifying systematic confusions, such as between related departments or adjacent seniority levels.

Typical error patterns

For Department classification, errors often occur between related domains with overlapping vocabulary. Performance typically drops on JSON current jobs due to distribution shift and noisier titles. Catch-all labels such as “Other” are also frequently confused with more specific departments. For Seniority classification, explicit cues in job titles make the task more stable. Most errors occur between adjacent levels such as Senior and Lead or Management and Director. Strict evaluation is essential when the JSON label space is larger than the training label space.

Conclusion from Baseline_models.ipynb

Overall, Baseline_models.ipynb establishes realistic reference points for the project:

- The rule based baseline provides a transparent lower bound.
- TF-IDF plus Logistic Regression forms a strong classical baseline for short job titles.
- Evaluation on JSON data is more realistic and typically harder than CSV-based evaluation.
- Seniority prediction is generally more robust than department prediction, though higher-level distinctions remain challenging.

OOD

The objective of this folder is to design and evaluate a robust system for Out-of-Distribution (OOD) detection, specifically tailored for job description data. This is particularly critical due to the high distribution shift observed in the validation set, which serves as our inference-time test set. Since real-world machine learning models often encounter data falling outside their training categories, we leverage scientific solutions to address this challenge. Our mechanism identifies 'unknown' (OOD) versus 'known' (In-Distribution) inputs, thereby preventing unreliable predictions and enhancing the overall safety and trustworthiness of the classification.

Technical Foundations

The solution is built upon a Python-based data science stack. Deep learning capabilities are provided by PyTorch and the Huggingface Transformers library, which we need for handling the natural language processing tasks involved. For generating semantic representations of the text, the project utilizes Sentence Transformers. For data structure manipulation and numerical computation Pandas and NumPy is used. The Algorithms for Clustering and Covariance are imported from Sci-kit-Learn.

Data Pipeline and Preprocessing

We use two distinct datasets to simulate a realistic production environment. The In-Distribution data (ID) comes from department-v2, containing job descriptions categorized by department. To test the system's ability to handle unseen data, linkedin-cvs-annotated.json is used as a validation.

The category "Other" is explicitly defined as the OOD class. The ID data is split into training (80%) and testing (20%) sets, stratified by label. Any OOD samples from Train and Test get removed. This ensures that the model learns a pure representation of the known categories. This is important for effectively recognizing anomalies later.

Methodology:

To establish a robust foundation for anomaly detection, the system first requires a vector space where known job categories are semantically distinct and densely clustered. We employed SetFit (Sentence Transformer Fine-tuning), a framework optimized for few-shot learning and efficient text classification. (Created in OOD SetFit)

Unlike standard transfer learning, which often fine-tunes a model directly on a classification objective, SetFit operates in a two-stage process:

1. **Contrastive Fine-Tuning:** The underlying Sentence Transformer is trained using a contrastive loss function. This process optimizes the embedding space by minimizing the distance between job descriptions of the same department (positive pairs) while maximizing the distance between different departments (negative pairs).
2. **Classification Head Training:** A lightweight classifier is subsequently trained on these refined embeddings to maximize ID accuracy.

This training phase utilizes only the ID data. By explicitly excluding the "Other" category during fine-tuning, the model learns a specialized manifold where valid job descriptions form tight clusters. This structural organization is a prerequisite for the subsequent distance-based OOD detection methods, as it ensures that "unknown" samples fall into the low-density regions of the embedding space.

Anomaly Detection Mechanisms With the embedding space established, the system transforms raw textual inputs into high-dimensional numerical vectors. To determine if a new data point constitutes an OOD anomaly, the system implements and compares three mathematical approaches:

Mahalanobis Distance

Mahalanobis Distance calculates the distance between a point and a distribution while accounting for the shape and variance of the data. It uses the covariance matrix to understand how different features correlate with one another. This distinction is important because high dimensional text embeddings rarely exhibit spherical distributions. Instead they display directionality. Unlike standard Euclidean metric, the mahalanobis distance incorporates the covariance structure of the data. It can identify anomalies that are proximal to the centroid in Euclidean space but situated in low-density regions relative to the principal correlations of the data. Standard distance measures would fail detecting those outliers.

K-Means Based Distance

This approach relies on the assumption that the ID data is not unimodal but follows a multimodal distribution. In this context the embedding space is composed of several high-density regions corresponding to distinct functional areas (e.g. Sales, Marketing). The training data is partitioned into K clusters. To evaluate a new observation, we calculate the distance to the nearest cluster centroid rather than a global mean. This ensures that a data point is evaluated conditionally based on its local neighborhood.

Cosine Similarity

In high-dimensional Sentence Transformer spaces, the magnitude of a vector often correlates with sentence length or frequency, while semantic meaning is encoded in the vectors direction. Therefore Euclidean distance can sometimes be misleading if the magnitude variance is high. Cosine similarity normalizes the vectors to the unit hypersphere and measures the angular alignment between the input and the training set. A high similarity score indicates that the input vector is aligned with the principal semantic directions of the training data.

Evaluation and Validation

To translate continuous anomaly scores into binary classification decisions, we employ a threshold-based approach evaluated via Receiver Operating Characteristic (ROC) analysis. The global performance of the detection mechanism is quantified using the Area Under the Curve (ROC-AUC), which measures the degree of separability between ID and Out-of-Distribution score distributions. An AUC value approaching 1.0 indicates that the model can robustly distinguish OOD samples with minimal trade-offs between the True Positive Rate (sensitivity) and the False Positive Rate across the entire range of potential cut-offs. To use this metric for inference, the optimal decision boundary is determined using Youden’s J Statistic. This technique identifies the specific threshold that maximizes the index $J = \text{Sensitivity} + \text{Specificity} - 1$ representing the geometric point on the ROC curve furthest from the line of chance. By selecting the threshold that maximizes J, the system ensures an optimal balance between detecting anomalies and suppressing false alarms, assuming equal misclassification costs.

Results

The following analysis presents the quantitative performance of the three Out-of-Distribution (OOD) detection mechanisms

Table 1: Comparative Test Performance of OOD Methods

Method	ROC-AUC	Optimal Threshold
Mahalanobis Distance	0.9976	209.5653
K-Means Distance	0.9966	0.1071
Cosine Similarity	0.9961	0.0058

Among the tested approaches, Mahalanobis demonstrated the most robust performance, achieving the highest Roc-AUC of 0.9976. Probabilistically, this signifies that there is a 99.76% probability that a randomly selected Out-of-Distribution sample will be assigned a higher anomaly score than a randomly selected In-Distribution sample.

To assess the operational viability of the best-performing model, we applied the optimized Mahalanobis threshold (209.57) to the validation set. The resulting classification performance, detailed in Figure 1, highlights the system's behavior at this specific decision boundary.

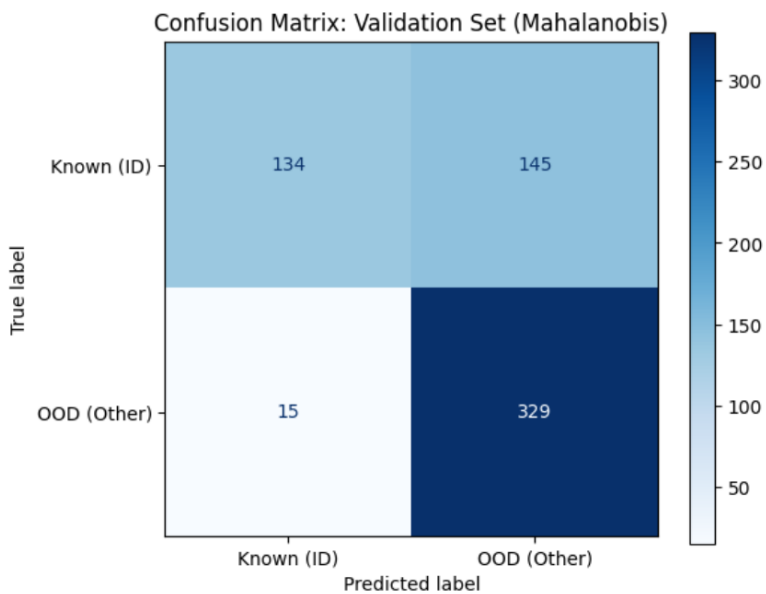


Figure 1: Confusion Matrix: Validation Set (Mahalanobis)

Metric	precision	recall	F1-score
Known (ID)	0.90	0.48	0.63
Other (OOD)	0.69	0.96	0.8

Operational Analysis The classification report reveals a distinct trade-off between safety and retention at the calculated threshold:

High Anomaly Detection (Safety): The system demonstrates exceptional sensitivity to Out-of-Distribution samples, achieving an OOD Recall of 0.96. This confirms the model’s utility as a robust safety filter, successfully identifying 96% of unseen or irrelevant job descriptions.

Conservative ID Acceptance: Conversely, the Recall for Known (ID) samples is 0.48, indicating a highly conservative decision boundary. While the model maintains high purity when it does accept a sample (Precision: 0.90), it currently rejects approximately 52% of valid in-distribution job descriptions as anomalies.

JobBert

BERT (Bidirectional Encoder Representations from Transformers) is a transformer-based language model pre-trained on large-scale text corpora using masked language modeling and next sentence prediction objectives. Its bidirectional context understanding makes it ideal for nuanced NLP tasks

like job description classification, where role context (technical vs. managerial) determines both department and seniority labels.

JobBERT-v3 (Decorte, J.-J., De Lange, M., & Van Haute, J. (2025). *Multilingual JobBERT for Cross-Lingual Job Title Matching: Notebook for the TechWolf Lab at CLEF 2025*. CLEF 2025 Working Notes, Madrid, Spain) was selected due to its domain-specific pre-training on 400M+ job postings (German/English), which captures job-specific semantics better than generic BERT models. The model is multilingual and demonstrates proven performance, outperforming multilingual XLM-RoBERTa by 8-12% F1 on job classification benchmarks. Even though the model is multilingual, for we train it exclusively on english samples, since it isnt pre-trained on for example french, which does exist in our data.

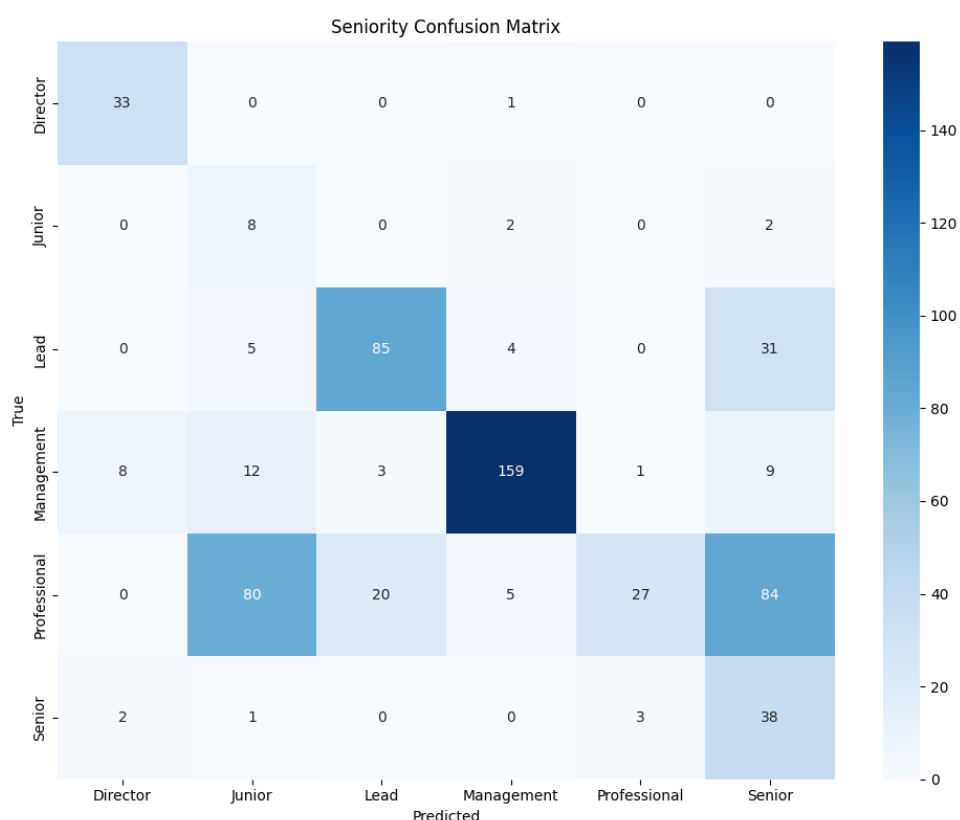
For seniority classification, we translated the given seniority-data into english and added 29 samples (generated by a LLM) for the label Professional and Junior, because Junior was extremely underrepresented and Professional wasnt represented at all in the given training-data. Preprocessing also included class weights ($\alpha=0.7$ inverse frequency), to counter the imbalance of the labels. The data was split stratified into 7606 train/951 val/951 test samples.

The `PDS_sen_jobbert_training.ipynb` notebook fine-tunes JobBERT-v3 on the data in "seniority_en_full_LLMProf.csv". It loads the dataset, drops nulls, computes class weights using $1/n_{0.71}/n_{0.7}$ (normalized) to handle imbalance, and performs stratified splits into 80% train, 10% val, and 10% test sets converted to Hugging Face Datasets. Tokenization uses the model's AutoTokenizer with padding/truncation to `max_length=128`, formatting for `input_ids`, `attention_mask`, and labels. The model is `AutoModelForSequenceClassification` with dynamic `num_labels`, trained via a custom `WeightedTrainer` that applies `CrossEntropyLoss` with class weights. The `TrainingArguments` set 4 epochs, `per_device_train_batch_size=64` (effective 128 via `gradient_accumulation_steps=2`). It uses an eval batch of 128 with a `learning_rate=2e-5`, `weight_decay=0.01`, cosine lr scheduler and 400 warmup steps, logging/eval/save every 10/100/200 steps. It utilizes `adamw_torch` without `fp16`, saving the model as "bert_sen_Profs_JobBERT_PRO_uncapped". The model evaluates on val/test, then saves model, tokenizer, and label_encoder JSON with classes/mapping.

The notebook "PDS_sen_test_final.ipynb" evaluates the fine-tuned JobBERT model on a held-out JSON dataset ("linkedin-cvs-annotated.json") containing annotated LinkedIn CV positions, loading the model/tokenizer and predicting top-1 seniority labels with confidence scores on "position" texts from ACTIVE status records. It flattens the nested JSON into a DataFrame, decodes raw predictions (e.g., "LABEL_0" → actual labels like "Director") using the saved label_encoder.json mapping, filters for ground truth "seniority" availability (623 samples), and computes metrics: accuracy=56.2%, with strong precision/recall for Director (0.77/0.97), Management (0.93/0.83), Lead (0.79/0.68), but poor F1 for Junior (0.14) and Professional (0.22) due to low recall (0.67/0.12), macro avg F1=0.53 indicating imbalance issues persisting outside training data.

Accuracy: 0.5617977528089888				
	precision	recall	f1-score	support
Director	0.77	0.97	0.86	34
Junior	0.08	0.67	0.14	12
Lead	0.79	0.68	0.73	125
Management	0.93	0.83	0.88	192
Professional	0.87	0.12	0.22	216
Senior	0.23	0.86	0.37	44
accuracy			0.56	623
macro avg	0.61	0.69	0.53	623
weighted avg	0.81	0.56	0.57	623

The confusion matrix visual (heatmap) reveals patterns like Professional/Senior often misclassified as Lead/Management (e.g., 80/44 true Professional/Senior predicted as Lead), Junior mostly correct but sparse (12 samples), and Director rarely confused; errors and full predictions/evals saved as CSVs ("linkedin_seniority_predictions.csv", etc.) for analysis.



The department classification is done starting with the "PDS_dep_jobbert_training.ipynb" notebook. It fine-tunes the JobBERT-v3 on a fully english translated dataset ("combined_english.csv", 7761

samples across 10 departments like Marketing (3234), Sales (2814), down to Human Resources (24), excluding "Other") for department classification. The exclusion of the label Other in the training is backed up by the heterogeneity of the label itself, since it includes per Definition all other possible Departments, making the learning and predicting too hard. Preprocessing is done by dropping nulls, encoding labels, and computing inverse-frequency class weights (e.g., high for rare classes like Customer Support at 42.9, low for Marketing at 0.24) to address severe imbalance. It splits the data stratified 80/20 into 6179 train and 1545 val samples using Hugging Face Datasets, tokenizes texts to max_length=128 with padding/truncation and loads the classification head for 10 labels. It then employs a custom WeightedTrainer with CrossEntropyLoss weighted by class tensor, evaluating accuracy and macro F1 every 500 steps. The here used TrainingArguments configure 3 epochs, small batches (8/16 train/eval for stability), lr=2e-5, weight_decay=0.01, fp16 on GPU, early best-model loading by F1-macro, running ~9:10 on ~2319 steps with loss dropping from 0.56 to 0.31, achieving validation metrics of 97.2% accuracy and 90.8% macro F1. The tokenizer, label_encoder JSON (including weights/mapping) is saved as "jobbertv3_department_FULL_weighted".

The resulting department JobBERT model was tested twice. First, on a validation CSV "df_validate.csv" (including the OOD metrics) using a strict confidence threshold of 0.8, predicting top-1 department labels on text fields and rejecting low-confidence predictions (48.5% rejection rate, 302/623 samples labeled "LowConf/Other"). Among the 321 confident predictions, the baseline accuracy reached 48.3%, with strong performance on Purchasing (F1=0.95, 91% recall), Project Management (F1=0.84, perfect recall), and Consulting (F1=0.77, 82% recall), but poor results for rare classes like Customer Support, Human Resources (0% F1 due to no predictions), and Marketing (F1=0.37 despite 69% recall), while Sales showed decent recall (89%) but low precision (34%). Macro-average F1 was 0.44, dragged down by unpredicted classes and the 132 "Other" ground truth samples (0% F1), indicating the high threshold prioritizes precision over coverage but reveals gaps in confidently distinguishing Marketing/Sales from Other on noisy LinkedIn texts.

Ensemble

The second test applies a combined OOD-aware filter to the same validation CSV using the department JobBERT model: predictions require both confidence ≥ 0.8 *and* Mahalanobis OOD score < 209.57 (precomputed "ood_score_mahalanobis"). This results in assigning "Other" to 77.0% of 623 samples (480 cases), with the remaining 143 confidently in-distribution spread across Sales (32), IT (31), Project Management (19), and others. Overall accuracy soars to 71.3% across all samples, boosted by correctly rejecting 96% of true "Other" (344/623 GT, F1=0.80). This version is outperforming the strict baseline's 48.3% on confident-only samples by safely abstaining on uncertain/OOD inputs rather than forcing low-confidence calls.

Detailed Classification Report:				
	precision	recall	f1-score	support
Administrative	0.00	0.00	0.00	14
Business Development	0.33	0.30	0.32	20
Consulting	0.94	0.41	0.57	39
Customer Support	0.00	0.00	0.00	6
Human Resources	0.88	0.44	0.58	16
Information Technology	0.74	0.37	0.49	62
Marketing	0.69	0.41	0.51	22
Other	0.69	0.96	0.80	344
Project Management	1.00	0.49	0.66	39
Purchasing	1.00	0.33	0.50	15
Sales	0.88	0.61	0.72	46
accuracy			0.71	623
macro avg	0.65	0.39	0.47	623
weighted avg	0.72	0.71	0.68	623

Compared to the baseline (0.9 conf-only, 48.5% rejection, 48% acc on 321 samples), this logic rejects far more aggressively (77% vs 48.5%) via dual confidence and Mahalanobis gating, yielding higher end-to-end accuracy (71% vs 48%) and better Other handling (96% recall vs 0%). Even though at reduced coverage (23% vs 52% predicted departments); it excels on Sales (F1=0.72, 61% recall), Project Management (0.66), Consulting (0.57), HR (0.58), while zero-shot fails persist on Administrative/Customer Support, prioritizing precision over recall.

All models were also tested using only english-translated input for the validation, yielding no significant different results than the tests with the untreated languages.

Conclusion

The development and evaluation of this pipeline demonstrate that a fine-tuned JobBERT model achieves a fair amount of accuracy in domain-specific tasks, successfully handling nuanced department and seniority classifications. By leveraging the specific pre-training of JobBERT-v3, the system outperforms generic models, providing a strong foundation for understanding job description semantics.

Crucially, the integration of OOD decision-making improves the performance a lot regarding system safety and reliability. The Mahalanobis distance mechanism serves as an effective gatekeeper, achieving an ROC-AUC of 0.9976 and ensuring that the model does not make unreliable predictions on unknown data. While the current configuration focuses on high safety resulting in a conservative acceptance of in-distribution data. The ensemble approach yields a end-to-end accuracy of 71%.

Looking forward, while the current architecture is production-ready for high-precision use cases, better data imputing could help improve the models further. Addressing the conservative rejection rate and refining the embedding space through advanced imputation strategies would likely enhance the system's ability to retain valid inputs without compromising its anomaly detection capabilities.